



TV2 – Hyena Method

Long Convolution + Gating thay the Self-Attention

Tô Huỳnh Minh Tiến · Nhóm 08

Hyena Hierarchy, Poli et al., ICML 2023



Câu hỏi TV2 cần trả lời: Hyena thay thế self-attention bằng toán tử nào, và vì sao toán tử đó rẻ hơn khi context dài?

Attention làm tốt	Vấn đề khi L dài	Hyena thay bằng
Trộn thông tin toàn chuỗi	Mã trận $L \times L$	Long convolution
Chọn lọc theo nội dung input	$O(L^2)$ time/memory	Data-controlled gating
Chất lượng LM mạnh	Khó mở rộng long-context	FFTConv $O(L \log L)$

TV1: Attention bottleneck → TV2: Hyena operator → TV3: small reproduction



Hyena = Long Convolution + Data-Controlled Gating

Long convolution

- Filter dài gần bằng sequence.
- Mang tín hiệu từ token xa.
- Tính nhanh bằng FFTConv.

Gating

- Sinh từ projection của input.
- Nhân element-wise với output conv.
- Tạo tính content-aware.

Input u

-> linear projections: $x_1, x_2, \dots, v$

-> $z_1 = v$

-> repeat: $z(n+1) = \text{gate } x_n * \text{Conv}(h_n, z_n)$

-> output y

Câu nói ngắn: convolution truyền thông tin xa, gating quyết định giữ thông tin nào.



Từ local kernel sang full-context filter

Mô hình	Kernel/filter	Phạm vi nhìn
CNN thường	Ngắn, local	Vài token lân cận
Hyena	Dài bằng sequence	Toàn bộ context

$$(h * u)_t = \sum_{i=0}^t h_{t-i} u_i$$

CNN local: token t chỉ nhận từ vùng gần $[x \ x \ x] \text{ ---- } t$
Long conv: token t có thể nhận từ rất xa $[x \ x \ x \ x \ x \ x \ x \ x \ x] \ t$

Ý nghĩa: long convolution giúp mô hình hóa phụ thuộc xa mà không cần ma trận attention $L \times L$.



Vì sao cần gating?

- Convolution thuần thường khá **tĩnh**: cùng filter áp dụng cho mọi input.
- Language modeling cần chọn lọc theo **ngữ cảnh input**.
- Hyena dùng gate x^n được chiếu từ input, rồi nhân với output convolution.

conv output:

[0.8, 0.4, 0.9, 0.2]

gate x:

[1.0, 0.1, 0.7, 0.0]

selected signal:

[0.8, 0.04, 0.63, 0.0]

gate cao $\rightarrow$ giữ

gate thấp $\rightarrow$ giảm

Long convolution đưa thông tin từ xa về; gating quyết định thông tin nào nên được giữ lại.



Công thức trung tâm

$$z_t^{n+1} = x_t^n \cdot (h^n * z^n)_t$$

Thành phần	Ý nghĩa
$z^1 = v$	value stream ban đầu
h^n	long filter ở bước n
x^n_t	gate tại token t
N	số bậc/order

Output cuối: $y = z^{(N+1)}$.

Bước n :

1. lấy $z(n)$
2. Conv với filter $h(n)$
3. nhân gate $x(n)$
4. ra $z(n+1)$



Vì sao gọi là "Hierarchy"?

- Hyena có thể lặp nhiều bước **convolution + gating**.
- **N** càng lớn, operator càng biểu diễn phong phú hơn.
- Các cấu trúc như H3/GSS có thể xem là liên quan ở bậc thấp.
- Trong repo nhóm: Hyena-small dùng **order = 2** để dễ reproduction.

Order 1: $z1 \rightarrow \text{Conv } h1 \rightarrow \text{Gate } x1 \rightarrow z2$

Order 2: $z1 \rightarrow \text{Conv } h1 \rightarrow \text{Gate } x1 \rightarrow z2 \rightarrow \text{Conv } h2 \rightarrow \text{Gate } x2 \rightarrow z3$

Order N: repeat N lần $\rightarrow$ output

Không cần chứng minh hierarchy; chỉ cần nắm: mỗi order thêm một tầng tương tác có cấu trúc.



Trực giác ma trận

Thành phần Hyena	Dạng ma trận	Trực giác
Gating x^n	Diagonal matrix D_x	nhân từng vị trí/channel
Convolution $h^n * z^n$	Toeplitz matrix S_h	trượt cùng một filter qua chuỗi

Hyena xen kẽ D_x và S_h , thay vì tạo một attention matrix dense $L \times L$.

Attention: $y = A(x) \cdot v$ A là dense $L \times L$
 Hyena: $y \approx D_{x2} \cdot S_{h2} \cdot D_{x1} \cdot S_{h1} \cdot v$

Điểm chính: vẫn phụ thuộc input qua D_x , nhưng tận dụng cấu trúc để tính rẻ hơn.



Filter dài nhưng ít tham số

$$h_t = \text{Window}(t) \cdot \text{FFN}(\text{PE}(t))$$

Cách học filter	Ý tưởng	Vấn đề/lợi ích
Explicit	lưu trực tiếp <code>h[0...L-1]</code>	dài hơn thì thêm tham số
Implicit	học hàm sinh <code>h_t</code> từ vị trí <code>t</code>	tham số nằm trong FFN

position `t` -> Positional Encoding -> small FFN -> raw filter value
raw value * Window(`t`) -> `h_t`

Window/decay giúp filter ổn định hơn ở các khoảng cách xa.

Tính long convolution hiệu quả

Ý tưởng: chuyển convolution sang miền tần số để phép convolution trở thành phép nhân element-wise.

1. FFT(h) và FFT(u)
2. Nhân element-wise trong miền tần số
3. iFFT để quay lại sequence output

Cách tính	Chi phí trực giác	Khi nào quan trọng
Direct convolution	tốn hơn khi filter rất dài	ít hợp với long-context
FFTConv	khoảng $O(L \log L)$	lợi khi L lớn

Trong repo: `models/hyena.py` -> `HyenaOperator._causal_fft_conv`.



Điểm rơi của Hyena là context dài

Method	Time	Memory
Standard Attention	$O(L^2)$	$O(L^2)$
FlashAttention	$O(L^2)$ compute	tối ưu memory access
Hyena	$O(N \cdot L \log L)$	gần tuyến tính theo L

Khi L tăng rất lớn, $L \log L$ tăng chậm hơn L^2 , nên lợi thế của Hyena rõ nhất ở long-context.

Lưu ý khi nói: Hyena không nhất thiết nhanh hơn ở L nhỏ vì FFT có overhead.

Paper chứng minh 2 thứ: quality và efficiency

Quality

Benchmark	Kết quả
WikiText-103	Hyena-3 ~ Transformer 125M
The Pile 335M	Hyena-2 gần Transformer
Synthetic tasks	tốt trên recall/reasoning dài

Efficiency

Length	Kết quả runtime
2K	gần crossover
8K	~5x vs attention, ~2x vs FlashAttention
64K	>100x trong paper

Đây là kết quả của **paper gốc**, không phải kết quả reproduction của nhóm.



Chuyển Sang Reproduction

Các kết quả vừa rồi là của paper gốc ở quy mô lớn.

Trong phạm vi môn học, nhóm thu nhỏ bài toán để kiểm chứng xu hướng trên WikiText-2 với Transformer-small và Hyena-small.